

Microservices are exposed to similar threats as web-based applications (e.g., cross-site scripting or injection attacks). Controls to prevent or mitigate these attacks should be included in the microservices code.⁶² Microservices can have security, reliability, and latency issues. Having multiple microservices can increase the entity's attack surface. Effective management evaluates implementation options that meet the entity's security requirements. Management of microservices-based architectures should include monitoring services, which may be running on different servers or written in different languages. Management should consider the following monitoring controls:

- Monitoring at the gateway and service level.
- Implementing a centralized dashboard to display the status of multiple services and network segments.
- Creating a baseline and implementing intrusion detection to provide alerts on deviations from the baseline.

To preserve resilience when developing and using microservices, NIST recommends the following control examples:⁶³

- Load balancing by having multiple instances of the same service and evenly distributing the load of those instances.
- Implementing circuit breaking by setting a threshold for failed responses from a microservice. When the threshold is exceeded (i.e., the circuit breaker is tripped), requests should not be forwarded to that microservice. The goal is to prevent cascading failures and allow time to investigate and address the issue.
- Throttling or limiting the rate of requests to a microservice.
- Redirecting requests to new versions of the microservice.⁶⁴
- Limiting traffic to new versions of a microservice until management can validate the correctness of a response and understand the performance.⁶⁵

For more information on the development and use of microservices, refer to the "[Development](#)" section of this booklet.

⁶² For more information on specific controls, refer to "[OWASP API Security Project](#)."

⁶³ Refer to NIST SP 800-204A, [Building Secure Microservices-Based Applications Using Service-Mesh Architecture](#).

⁶⁴ This is referred to as blue/green deployments. When a new version of a microservice is deployed, requests from customers using the old version can be redirected to the new version using the API gateway that can be programmed to maintain awareness of the locations of both versions. For more information, refer to NIST SP800-204A, [Building Secure Microservices-Based Applications Using Service-Mesh Architecture](#).

⁶⁵ This process is referred to as "canary releases," in which only a limited amount of traffic is initially sent to a new version of a microservice because the correctness of its response or performance metric under all operating scenarios is not fully known. Once sufficient data are gathered about its operating characteristics, the requests can be proxied to the new version of the microservice. For more information, refer to NIST SP 800-204A, [Building Secure Microservices-Based Applications Using Service-Mesh Architecture](#).

IV.H Containers

A container is a method for packaging and securely running an application in a virtualized environment. Containers enable applications to be portable, reusable, and automatable in terms of their creation, use, and destruction. Containers provide software developers the ability to segment applications in separate environments to support well-defined functionality. This allows the entity to coordinate the development, testing, implementation, and operation of applications in a more consistent, precise, and efficient manner. Containers enable virtualized segmentation, which allows for increased security and transparency but may require additional configuration to support effective integration. In addition to the use of containers in the production environment, it is beneficial to leverage containers in the development and test environments, which helps to promote consistency.

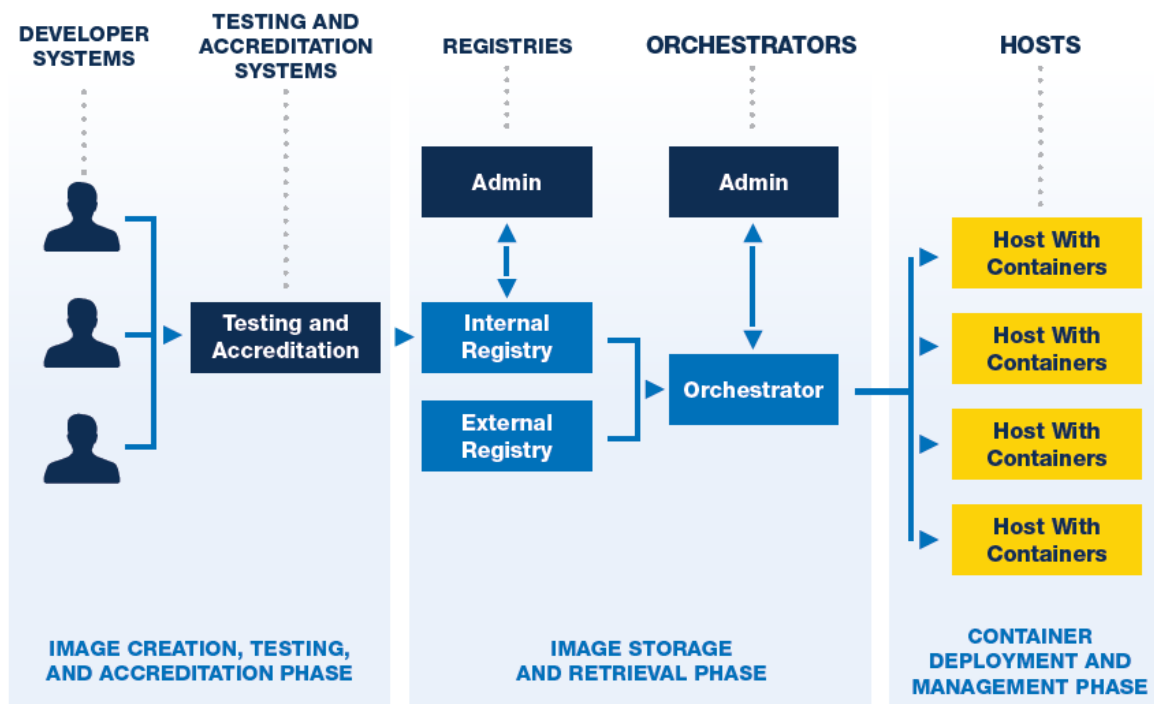
Figure 1 depicts the following five foundational architectural tiers and components and the three life cycle phases for container development and technology architecture.⁶⁶ The foundational architectural tiers and components are as follows:

- Developer systems, in the image creation, testing, and accreditation phase, generate container images and send them for testing and accreditation.
- Testing and accreditation systems, in the image creation, testing, and accreditation phase, validate and verify the contents of container images, sign container images, and send container images to the registry.
- Registries (internal and external), in the image storage and retrieval phase, store images and distribute container images to the orchestrator on request.
- Orchestrators facilitate the conversion of container images into containers during the image storage and retrieval phase and deploy containers to hosts in the container deployment and management phase.
- Hosts run and stop containers as directed by the orchestrator in the container deployment and management phase.

The three phases noted in figure 1 are (1) image creation, testing, and accreditation; (2) image storage and retrieval; and (3) container deployment and management for container development and technology architecture.

⁶⁶ For more information, refer to NIST SP 800-190, [Application Container Security Guide](#).

Figure 1: Container Technology Architecture Tiers and Components and Life Cycle Phases



When setting up containers, it is important to consider data isolation. Data may be stored separately from configuration information and software (i.e., as separate building blocks). When data and configuration information are stored separately from the containerized software, the software, data, or configuration information can be created, used, and destroyed without consequence to the other building blocks in the container. Management may use persistent storage⁶⁷ for ongoing access to containerized applications, data, and configuration information. The aspects of isolation and persistent storage allow for new versions of containers to be created and used efficiently, without losing previously used data. When a new version of the software or containerized image is executed or initialized, connections to the persistent storage need to be restored along with any authorization and access rights considerations.

Container images are typically designed to be portable across machines and environments so that an image created in a development lab can be easily moved to a test lab for evaluation, then copied into a production environment to run without needing to make any modifications.⁶⁸ This deployment process (i.e., copying the image into production) results in a version of the application that is running and available to respond to requests. According to NIST, “When an image is deployed into a container, the image itself is not changed, but instead a copy of it is

⁶⁷ Merriam-Webster defines “[persistent](#)” as “continuing without change in function or structure.” For purposes of this discussion regarding storage in containers, “persistent storage” refers to the long-term access to application information beyond the container’s lifetime.

⁶⁸ Refer to NIST SP 800-190, [Application Container Security Guide](#).

placed in the container and transitioned from being a dormant set of [application] code to a running instance of the [application].”⁶⁹

The use of containers is not without risks; therefore, during container development, management should consider the risks and countermeasures provided in NIST 800-190,⁷⁰ such as those in table 1.

Table 1: Container Risks and Countermeasures

Risks	Countermeasures
Image risks	
Image vulnerabilities ⁷¹	Consider containers and images when selecting or using vulnerability management tools.
Image configuration defects	Adopt tools and processes to validate and enforce compliance with secure configuration best practices, such as implementing privileged user controls and validation of image configuration settings. ⁷²
Embedded malware	Monitor images for embedded malware by monitoring for malware signature sets and detecting behavioral patterns and anomalies.
Clear text secrets ⁷³	Use orchestrators and APIs to manage security of access to and provision of secrets to specific containers that require them based on predefined settings.
Untrusted images	Maintain a set of trusted images and registries. Only use images from this set to run in the environment, thus mitigating the risk of untrusted or malicious components being deployed. Consider processes to ensure that images and registries are secure, trusted, and periodically validated and that the hosts they reside on are from these approved lists. ⁷⁴
Registry risks	
Insecure connections to registries	Configure development tools, orchestrators, and container runtimes to connect only to registries over encrypted channels. The goal is to ensure that all data pushed to and pulled from a registry occur between trusted end points and are encrypted in transit.
Stale images in registries	Reduce registries to minimize unsafe and vulnerable images that should no longer be used. Access images using unchanging names that specify discrete versions of images to be used to ensure that specific and uncorrupted images are deployed as part of each job. ⁷⁵

⁶⁹ [Ibid.](#)

⁷⁰ [Ibid.](#)

⁷¹ Container image vulnerabilities may not be detected by traditional vulnerability management tools.

⁷² For more information, refer to NIST SP 800-190, [Application Container Security Guide](#).

⁷³ Many apps require secrets to enable secure communication between components. The risk is that if someone steals the image, that person can learn the secrets, such as username and password, to connect to a backend database, connection strings, and private keys. Refer to NIST SP 800-190, [Application Container Security Guide](#).

⁷⁴ For more information, refer to NIST SP 800-190, [Application Container Security Guide](#).

⁷⁵ Another option is using a “latest” tag for images and referencing this tag in deployment automation, but this may not always be effective. Regardless of whether an organization chooses to use discrete names or to use a “latest” tag, it is critical that processes exist to ensure that the most recent unique name or the images tagged “latest” represent

Risks	Countermeasures
Insufficient authentication and authorization restrictions	Any access to registries that contain proprietary or sensitive images should involve authentication. Gaining write-access privileges to a registry should entail appropriate authentication to ensure that only images from trusted entities can be added to the registry. Audit all write-access privileges to registries and log any read actions for sensitive images. ⁷⁶
Orchestrator Risks	
Access considerations, such as unbounded administrator access ⁷⁷	Orchestrators should use a least-privilege access model in which users are only granted the ability to perform the specific actions on the specific hosts, containers, and images that their job roles require. Test team members should have limited or no access to containers used in production. ⁷⁸
Unauthorized access	Tightly control access to cluster-wide administrative accounts, as these accounts provide the ability to affect all resources in the environment. Use appropriate authentication methods (e.g., MFA and single sign-on when applicable). Use tools for encrypting data used with containers that allow the data to be accessed properly from containers regardless of the node on which they are running. ⁷⁹
Orchestrator node trust ⁸⁰	Configure orchestration platforms to provide features that create a secure environment for all the applications they run. Maintaining a secure-by-default posture is particularly important with third-party deployments of the entity's containers. Orchestrators should be able to perform the following: introduce nodes securely to the cluster, ⁸¹ maintain a persistent identity for the nodes throughout their life cycle, and maintain an accurate inventory of nodes and their connectivity states. Orchestration platforms should be resilient (i.e., a compromise of one node should not compromise the overall security of the cluster). ⁸²
Poorly separated inter-container network traffic ⁸³	Configure orchestrators to separate network traffic into discrete (e.g., internal versus external applications) virtual networks by sensitivity level, and

the most up-to-date versions. For more information, refer to NIST SP 800-190, [Application Container Security Guide](#).

⁷⁶ For more information, refer to NIST SP 800-190, [Application Container Security Guide](#).

⁷⁷ A single orchestrator having administrator access. Access should be limited according to need. For example, applications may be managed by different teams with differing sensitivity levels. As many orchestrators were designed with the assumption that user interaction would be with administrators, there is the potential for unauthorized access; therefore, authorization and access concerns are relevant. Refer to NIST SP 800-190, [Application Container Security Guide](#).

⁷⁸ For more information, refer to NIST SP 800-190, [Application Container Security Guide](#).

⁷⁹ For more information, refer to NIST SP 800-190, [Application Container Security Guide](#), and the *FFIEC IT Handbook's* "[Information Security](#)" booklet.

⁸⁰ The orchestrator is the most foundational node. Weak orchestrator configurations can expose the orchestrator and other container technology components to increased risk. Refer to SP NIST 800-190, [Application Container Security Guide](#).

⁸¹ An example of a cluster is a set of nodes that run containerized.

⁸² A compromised node should have the ability to be isolated and removed from the cluster without disrupting or degrading overall cluster operations. For more information, refer to NIST SP 800-190, [Application Container Security Guide](#).

⁸³ If a virtual overlay network is used to manage container traffic, security and management tools may not have clear visibility into the traffic. If traffic has varying sensitivity levels on the same network, more sensitive data may be susceptible to attack due to expanded access to the network users with lesser privileges. For example, if the public-

Risks	Countermeasures
	communication between the two should occur through a small number of well-defined interfaces. ⁸⁴
Mixing of workload sensitivity levels on the same host ⁸⁵	Configure orchestrators to isolate deployments to specific sets of hosts by sensitivity levels. Implement rules that prevent high-sensitivity workloads from being placed on the same host as those running lower-sensitivity workloads. Segmenting containers by purpose, sensitivity, and threat posture provides additional defense-in-depth. ⁸⁶
Container Risks	
General container runtime-related risks ⁸⁷	Use tools or processes that continuously assess configuration settings across the environment and actively enforce them. Use segmentation to provide control and isolation over containers to minimize unauthorized access and loss of data. Run containers with the default secure computing profiles ⁸⁸ provided by their runtime. Consider using additional profiles for higher risk applications. Use tools to identify vulnerabilities in the container runtimes deployed to upgrade any instances at risk and to ensure that orchestrators only allow deployments to properly maintained container runtimes.
Unbounded network access from containers ⁸⁹	Employ a combination of network monitoring and filtering tools (e.g., application-aware tools) ⁹⁰ to mitigate the risk from inter-container traffic across networks of differing sensitivity levels.

facing website is compromised, attackers may be able to use shared networks to attack an internal app using sensitive information. Refer to NIST SP 800-190, [Application Container Security Guide](#).

⁸⁴ For more information, refer to NIST SP 800-190, [Application Container Security Guide](#).

⁸⁵ An orchestrator may place a container running a public-facing web server on the same host as one processing sensitive data, because that host happens to have available resources at the time of deployment, as the orchestrator manages scale and density of traffic. In the case of a critical vulnerability in the web server, a container processing sensitive data may be at greater risk of compromise due to the greater access to the host of the container. Refer to NIST SP 800-190, [Application Container Security Guide](#).

⁸⁶ Concepts such as application tiering and network and host segmentation should be taken into consideration when planning app deployments. By segmenting containers in this manner, it is much more difficult for an attacker who compromises one of the segments to expand that compromise to other segments. In larger-scale environments with hundreds of hosts and thousands of containers, this segmentation is automated to be practical. Common orchestration tools can help management in the segmentation process. For more information, refer to NIST SP 800-190, [Application Container Security Guide](#).

⁸⁷ Container runtimes include such risks as (1) insecure container runtime configurations and (2) capability for containers running in privileged mode to access host configurations, potentially compromising the host and all other containers on it.

⁸⁸ Secure computing profiles can be used to limit the system-level capabilities allocated to containers at runtime. For more information, refer to NIST SP 800-190, [Application Container Security Guide](#).

⁸⁹ By default, in most container runtimes, individual containers can access each other and the host OS over the network. If a container is compromised and acting maliciously, allowing this network traffic may expose other resources in the environment to risk. Tools and operational processes that are not container-aware are not able to inspect this traffic or determine whether it represents a threat. Refer to NIST SP 800-190, [Application Container Security Guide](#).

⁹⁰ Application-aware tools provide the ability of a system to recognize and classify applications passing through it. These tools allow systems to make decisions based on an application, its features, and the content it's carrying. These tools should be able to not just see the inter-container traffic but also dynamically generate the rules used to

Risks	Countermeasures
Rogue containers ⁹¹	Maintain separate environments for development, test, and production with specific controls (e.g., role-based access control) for container deployment to mitigate the potential for rogue containers. There should be a clear audit trail process during container development to give management information that associates any container build or modification to specific users.
Application vulnerabilities, such as risks inherent in the general use of applications	Implement additional container-aware tools ⁹² using behavioral learning (i.e., heuristics) and appropriate security profiles to detect anomalies and events, such as writing to unexpected locations and file types, sending traffic to unexpected network destinations, and storing or executing malware. Additionally, containers should be run with their root file systems in read-only mode to monitor for compromise, to isolate tampering to these specific locations, and separate them from the rest of the application.
Host OS Risks	
Large attack surface ⁹³	Minimize unnecessary functionality by using OSs designed to host only containers and have other services and functionality disabled to reduce the attack surface. Additionally, host OSs should have read-only file systems and have hardening practices implemented by default. Regularly scan and update the container runtime and lower-level components (e.g., kernel) in a timely manner.
Shared kernel ⁹⁴	Use separate, dedicated hosts for containerized and noncontainerized workloads. Isolating the workloads enables the application of appropriate safeguards to containers and avoids affecting noncontainerized workloads.
Improper user access rights, such as allowing users to log on directly ⁹⁵ to hosts to manage containers	Employ least privilege access. Audit all authentication to the OS, monitor all login anomalies, and log any privileged user activities to identify anomalous (and potentially unauthorized) access patterns.

filter this traffic based on the specific characteristics of the apps running in the containers. For more information, refer to NIST SP 800-190, [Application Container Security Guide](#).

⁹¹ Rogue containers are unplanned or unsanctioned containers in an environment. They may be used in development builds or test images and may pose additional risk to the organization, especially when they persist in the environment without the awareness of development teams and security administrators. Refer to NIST SP 800-190, [Application Container Security Guide](#).

⁹² Container-aware tools provide the ability to monitor the container environment and provide precise detection of anomalous and malicious activity within it. For more information, refer to NIST SP 800-190, [Application Container Security Guide](#).

⁹³ The larger the attack surface is, the better the odds are that an attacker can find and access a vulnerability, leading to a compromise of the host OS and the containers running on top of it. Refer to NIST SP 800-190, [Application Container Security Guide](#).

⁹⁴ The use of a shared kernel results in a larger attack surface as the level of isolation provided by container runtimes is not as high as that provided by hypervisors. Runtime container security means vetting all activities in the container application environment, from analysis of container and host activity to monitoring the protocols and payloads of network connections. Refer to NIST SP 800-190, [Application Container Security Guide](#).

⁹⁵ When users log on directly to the host with elevated privileges to manage specific containers, there is the possibility of accessing and affecting other containers on that host. Refer to NIST SP 800-190, [Application Container Security Guide](#).

Risks	Countermeasures
Host OS component vulnerabilities ⁹⁶	Use appropriate tools provided either by the OS vendor or other trusted organizations to regularly check for and apply updates to all software components used in the OS to mitigate potential vulnerabilities. This is particularly important for the kernel and container runtime components as newer releases of these components often add additional security protections and capabilities beyond simply correcting vulnerabilities. Deploy application components and their dependencies ⁹⁷ in a container to identify or prevent anomalies and configuration changes, promote resilience, and allow management to reduce its attack surface, prevent persistent storage of data on the host, promote statelessness ⁹⁸ on the host, and reduce application-level host dependencies.
Host OS file system tampering ⁹⁹	Run containers with the minimal set of file system permissions required. NIST recommends that containers should not be able to make sensitive directories available on a host's file system, especially those directories containing configuration settings for the OS. Monitor what directories are placed, made available, and accessed on the file system.

IV.I Application Programming Interfaces

An API is software code that allows two or more different programs to communicate with each other. APIs are an important part of various applications, including web, mobile, and software-as-a-service (SaaS) applications. APIs often link microservices and may be public, private, or a collaboration between customers, partners, and unaffiliated third parties to share information and allow their software to function together. Another function of APIs is to facilitate communication between containers. For more information on APIs and how they fit with other elements of an entity's IT environment, refer to the *FFIEC IT Handbook's* "[Architecture, Infrastructure, and Operations](#)" booklet.

When developing, acquiring, and maintaining APIs and related services, there are several considerations.¹⁰⁰ Examples include the following:

- Aligning the API with business processes, particularly when designing a microservice.
- Iterating the interface definition with developers before the service is used so that the resulting API serves as a contract between clients and services.

⁹⁶ All host OSs, even container-specific ones, provide foundational system elements, such as access and authentication. As a foundational element in container technology, host OS vulnerabilities can affect everything running on the host, including containers and apps. Refer to NIST SP 800-190, [Application Container Security Guide](#).

⁹⁷ In IT infrastructure, dependencies are relationships of reliance in and among infrastructure assets and systems that must be maintained for those systems to operate and provide services. For more information, refer to CISA's "[Infrastructure Dependency Primer](#)."

⁹⁸ According to NIST Glossary, statelessness or quality of being "[stateless](#)" refers to "a data representation or a process that is self-contained and does not depend on any external data store."

⁹⁹ A change to a sensitive system file on the host OS could affect the stability and security of the host and all other containers running on it. Refer to NIST SP 800-190, [Application Container Security Guide](#).

¹⁰⁰ For more information on developing, acquiring, and maintaining APIs, refer to NIST SP 800-204, [Security Strategies for Microservices-Based Application Systems](#).

- Determining and implementing appropriate security related to APIs.¹⁰¹

Risks related to APIs include the following:

- Inappropriate access controls.
- Inadequate user authentication.
- Excessive data exposure.
- Lack of resources for processing access requests, including rate limiting problems.
- Allowing modification of object properties.¹⁰²
- Misconfigured security.
- Injection flaws.
- Lack of appropriate ITAM and documentation leading to maintenance weaknesses and exposed end points.
- Lack of appropriate logging and monitoring.
- Lack of integration with the entity's incident response program.

IV.I.1 API Gateway

Understanding how APIs can act as a gateway to microservices is important. Unlike a legacy application where the endpoint may be only a single server, a microservices-based application consists of multiple endpoints. Therefore, a single-entry point (i.e., gateway) for all clients to access multiple component microservices of the application would allow for improved access. Another situation in which an API gateway may be deployed is as a front-end-to-backend¹⁰³ connector, as in the case of legacy enterprise software. This occurs when an entity migrates from a legacy enterprise application by gradually replacing its components with independent microservices over time. Direct communication between clients and multiple end points often results in excessive point-to-point connections; therefore, the API gateway's primary function is to route inbound requests to the correct downstream services to minimize the number of connections. All client requests first go through the API gateway, which then routes requests to the appropriate microservice. The API gateway often handles a request by invoking multiple microservices and aggregating the results. In some instances, the API gateway may be used as part of a back end for front-end services, which enables support for clients with different communication vectors (e.g., browser or mobile device), and then requests are divided into multiple gateways. Many API gateway use cases can support APIs written in different languages.¹⁰⁴

¹⁰¹ For more information on API security, refer to the *FFIEC IT Handbook's* "[Information Security](#)" and "[Architecture, Infrastructure, and Operations](#)" booklets. "[OWASP API Security Project](#)" also discusses API security issues and controls.

¹⁰² Object properties provide a simple association between name and value. All properties have a name, and value is one of the attributes linked with the property, which defines the access granted to the property.

¹⁰³ The front end of a system involves the parts that users see and interact with, whereas the system's back end includes the logic, data, and structure.

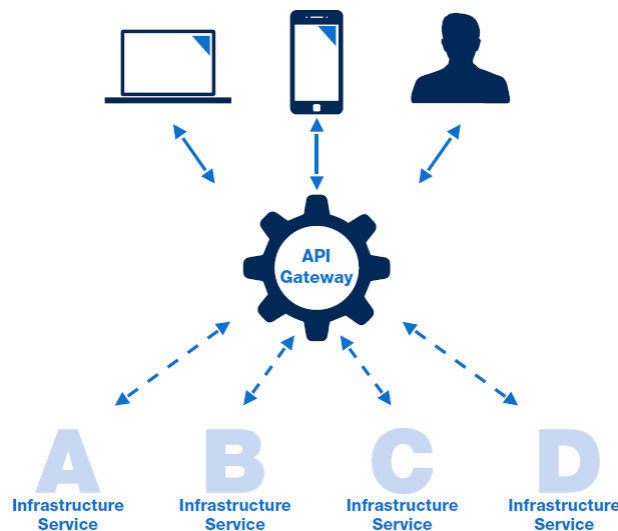
¹⁰⁴ Examples of different languages include Java, JavaScript, and Jolie.

Because the API gateway is the entry point for microservices, management should configure it with the necessary infrastructure services.¹⁰⁵ In addition to its main service of handling requests, an API gateway also helps with the following infrastructure services:

- Service discovery.
- Authentication and access control.
- Load balancing.
- Caching.
- Providing custom APIs for each type of client.
- Performing application-aware health checks.
- Service monitoring.
- Attack detection.
- Attack response.
- Security logging and monitoring.
- Circuit breaking.

These additional infrastructure services can be created or implemented in the code that uses the API gateway. Figure 2 provides a simple illustration of the API gateway location during a request. It shows that the API gateway routes requests from various sources (e.g., laptops, mobile devices, and users) to the various infrastructure services.

Figure 2: Location and Interaction of API Gateway



Microgateways are API gateways used to define and enforce customized policies for microservices-based applications. If used, they should be protected through service-specific security policies. Microgateways are usually implemented as containers and should contain policies for application requests and responses. When policies and their enforcement are implemented as a container, they are unchanging; therefore, they provide a degree of protection

¹⁰⁵ Refer to NIST SP 800-204, [Security Strategies for Microservices-Based Application Systems](#).